Chapitre 1
Partie 2- Gestion de l'intégrité

—

Dj BELKASMI (d.belkasmi@univ-boumerdes.dz) © 2024

Qu’est ce que l’intégrité ?

Se sont des règles et des conditions appliquées aux données qui assurent:

- ➔ leur validité,
- ➔ leur qualité,
- ➔ leur cohérence et
- ➔ leur précision

tout au long de leur cycle de vie.

Classes de CI

02 Catégories

CI Statiques

Elles sont définies lors de la création du schéma de la base de données.

CI Dynamiques

Elles s'appliquent en fonction des opérations effectuées sur les données.

Importance des CI

Leur importance réside dans leur capacité à :

- ➔ prévenir les erreurs de saisie,
- ➔ maintenir les relations entre les tables et
- ➔ garantir que les données respectent des règles spécifiques.

SGBD fiable et efficace.

Plan à suivre

- Les CI Statiques
  - Introduction
  - Types
  - Importance
  - Mise en oeuvre
- Les CI dynamiques
  - Introduction et définition
  - Types
  - Caractéristiques
  - Importance
  - Mise en oeuvre
- Conclusion

Les CI Statiques (CIS)

Types

Types	Contraintes	Explication
Entité	PRIMARY KEY	Définit une clé primaire qui identifie de manière unique chaque enregistrement.
	NOT NULL	Assure qu'une colonne ne peut pas contenir de valeurs nulles.
	UNIQUE	Garantit que les valeurs dans une colonne sont uniques.
Domaine	CHECK	Implique que les valeurs d'une colonne respectent des règles spécifiques.
Référentielle	FOREIGN KEY	Garantit que les valeurs d'une colonne dans une table correspondent aux valeurs d'une colonne dans une autre table

CI d'entité PRIMARY KEY

1. Identifie de manière unique chaque enregistrement dans une table.

2. Représente l'implémentation de la notion d'identificateur vu dans les entités.

3. Elle est capitale dans l'identification des lignes lors de la recherche.

Exemple d'implémentation en SQL

```
CREATE TABLE etudiants (
    matricule INT PRIMARY KEY
);
```

Introduction

1. Se sont des règles appliquées aux données d'une base de données qui garantissent que les données respectent certaines conditions avant d'être insérées ou mises à jour.

2. Se sont des contraintes qui s'appliquent à l'ensemble des données d'une table et qui ne changent pas au cours du temps.

3. Elles sont généralement définies lors de la création de la table et ne varient pas selon les transactions en cours.

CI d'entité NOT NULL

1. Empêche les valeurs nulles dans une colonne.
2. Elle est très importante dans les colonnes qui doivent toujours avoir une valeur.

   Comme exemple: nom, prénom, date_Naissance, etc...

Exemple d'implémentation en SQL

```
CREATE TABLE produits (
    id INT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL
);
```

CI d'entité UNIQUE

1. Elle garantit que que toutes les valeurs d'une colonne sont uniques (aucune valeur n'est en double).
2. Elle permet d'éviter qu'on puisse introduire la même valeur plusieurs fois.

Exemple d'implémentation en SQL

```
CREATE TABLE etudiants (
    matricule INT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE
);
```

CI référentielle FOREIGN KEY

1. Appelée clé étrangère est un concept fondamental qui permet de maintenir l'intégrité des données entre deux tables.
2. C’est une colonne (ou un ensemble de colonnes) dans une table qui établit un lien avec la clé primaire (PRIMARY KEY) d'une autre table.

CI de domaine CHECK

1. Elle permet de définir des conditions spécifiques que les valeurs d'une colonne doivent respecter.
2. Son utilisation dépendra de la nature des données ainsi que de la règle à appliquer.

Exemple d'implémentation en SQL

```
CREATE TABLE etudiants (
    matricule INT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    note INT CHECK (note BETWEEN 0 AND 20)
);
```

CI référentielle FOREIGN KEY

1. Ses objectifs:
    a. **Intégrité référentielle:** assure que les relations entre les tables sont valides. *Par exemple, si une commande fait référence à un client, ce client doit exister dans la table des clients.*
    b. Prévention des anomalies: empêche l'insertion de données "orphelines". *Par exemple, on ne peut créer une commande pour un client qui n'existe pas.*

CI référentielle FOREIGN KEY

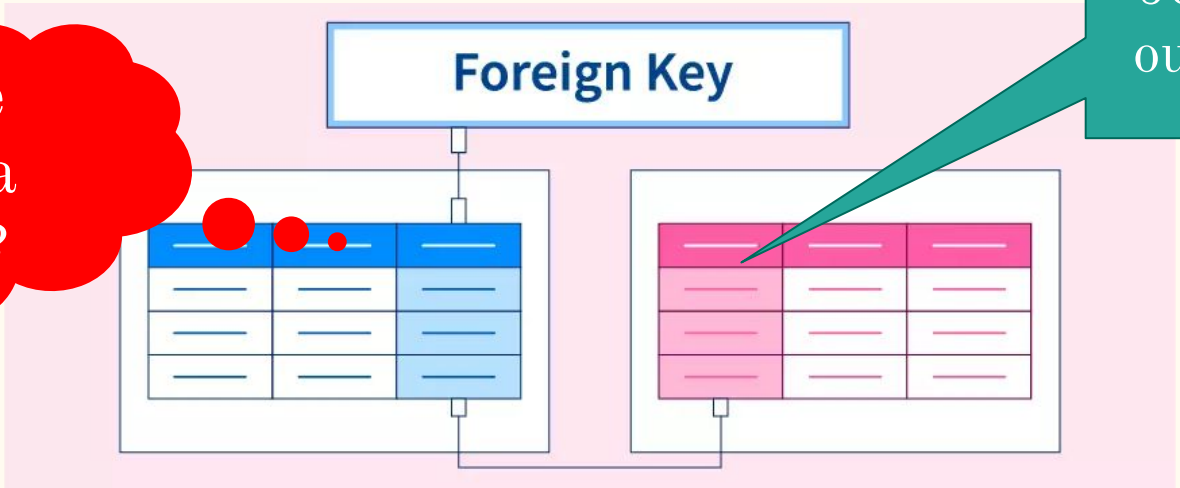
Exemple d'implémentation en SQL

```
CREATE TABLE etudiants (
    matricule INT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    note INT CHECK (note BETWEEN 0 AND 20),
    section_id INT,
    FOREIGN KEY (section_id) REFERENCES section(id)
);
```

CI référentielle FOREIGN KEY - Comportement

1. Comment se comporte le FOREIGN KEY lors des mises à jours (DELETE, UPDATE) ?

Que se passera t-il ici?



Je supprime ou je modifie

CI référentielle FOREIGN KEY - Comportement

Les comportement possibles:

- CASCADE
- SET NULL
- RESTRICT
- SET DEFAULT

CI référentielle FOREIGN KEY - Comportement

CASCADE:

1. Lorsque la ligne référencée dans la table parent est supprimée ou mise à jour, les lignes correspondantes dans la table enfant sont également supprimées ou mises à jour.
2. Utile lorsqu'on veut maintenir une cohérence totale entre les tables.

Exemple: si un client est supprimé, toutes ses commandes doivent également être supprimées.

Exemple d'implémentation en SQL
FOREIGN KEY (section_id) REFERENCES section(id) ON DELETE CASCADE

CI référentielle FOREIGN KEY - Comportement

SET NULL:

1. Lorsque la ligne référencée dans la table parent est supprimée ou mise à jour, la valeur de la clé étrangère dans la table enfant est définie sur NULL.
2. Pratique lorsqu'on souhaite conserver la ligne dans la table enfant, mais qu'on ne veut plus qu'elle soit associée à la ligne supprimée de la table parent.

Exemple d'implémentation en SQL
FOREIGN KEY (section_id) REFERENCES section(id) ON DELETE SET NULL

CI référentielle FOREIGN KEY - Comportement

RESTRICT:

1. Empêche la suppression ou la mise à jour de la ligne dans la table parent tant qu'il existe des références dans la table enfant.
2. Implique que si une ligne est référencée, on ne peut pas la supprimer ou la modifier.
3. Idéale pour garantir que les relations restent intactes et éviter toute perte de données.

Exemple d'implémentation en SQL
FOREIGN KEY (section_id) REFERENCES section(id) ON DELETE RESTRICT

CI référentielle FOREIGN KEY - Comportement

SET DEFAULT:

1. La valeur de la clé étrangère dans la table enfant est définie sur une valeur par défaut spécifiée si la ligne référencée dans la table parent est supprimée ou mise à jour.
2. L'idée est de remplacer la clé étrangère par une valeur par défaut au lieu de la laisser à NULL.

Exemple d'implémentation en SQL
FOREIGN KEY (section_id) REFERENCES section(id) ON DELETE SET DEFAULT

- Le comportement des FOREIGN KEY est crucial pour maintenir l'intégrité des données dans une BD.

- Le choix de la bonne option dépend des exigences spécifiques de votre application et de la manière dont vous souhaitez gérer les relations entre les tables.

Importance

1. **Maintien de l'Intégrité des Données:** garantissent que les données respectent certaines règles définies, ce qui aide à maintenir la cohérence et la fiabilité des informations stockées dans la BD.
2. **Prévention des Anomalies:** empêchent l'insertion de données invalides ou incohérentes.
3. **Facilitation des requêtes:** simplifie l'écriture de requêtes SQL. Les développeurs peuvent être assurés que les données auxquelles ils accèdent sont valides et conformes aux attentes.

Importance

4. **Amélioration des Performances:** améliorer les performances des requêtes en permettant à la base de données d'optimiser l'accès aux données. Par exemple, les index créés sur les clés primaires et étrangères peuvent accélérer les opérations de recherche.
5. **Sécurité des Données:** contribuent à la sécurité des données en empêchant les modifications non autorisées. Par exemple, une contrainte CHECK peut s'assurer que les valeurs d'une colonne respectent un certain format ou une certaine plage.

Importance

6. **Documentation des Règles Métier:** servent également de documentation implicite des règles métier au sein de la base de données. Elles décrivent les relations et les restrictions qui doivent être respectées, ce qui peut aider les développeurs et les administrateurs à comprendre la structure des données.

7. **Facilitation de la Maintenance:** Lorsqu'une BD est bien structurée avec des CI appropriées, elle est plus facile à maintenir. Les erreurs de données sont moins fréquentes, et les modifications peuvent être effectuées avec plus de confiance.

Mise en oeuvre

02 Façons

Lors de la création
CREATE TABLE

Après la création
ALTER TABLE

La contrainte est définie dans le script de création.

en deux étape:

- suppression de l'ancienne (**DROP**)
- ajout de la nouvelle (**ADD**)

Mise en oeuvre - lors de la création

Exemple d'implémentation en SQL

```sql
CREATE TABLE etudiants (
    matricule INT PRIMARY KEY
    nom VARCHAR(100) NOT NULL
    email VARCHAR(100) UNIQUE
    note INT CHECK (note BETWEEN 0 AND 20)
    NIN VARCHAR(18) NOT NULL
);
```

Mise en oeuvre - lors de la création

```
c:\wamp64_1\bin\mysql\mysql8.0.31\bin\mysql.exe
mysql> select * from information_schema.TABLE_CONSTRAINTS where TABLE_name='etudiants';
+--------------------+-------------------+-----------------+--------------+------------+-----------------+----------+
| CONSTRAINT_CATALOG | CONSTRAINT_SCHEMA | CONSTRAINT_NAME | TABLE_SCHEMA | TABLE_NAME | CONSTRAINT_TYPE | ENFORCED |
+--------------------+-------------------+-----------------+--------------+------------+-----------------+----------+
| def                | i3_bda            | email           | i3_bda       | etudiants  | UNIQUE          | YES      |
| def                | i3_bda            | PRIMARY         | i3_bda       | etudiants  | PRIMARY KEY     | YES      |
| def                | i3_bda            | etudiants_chk_1 | i3_bda       | etudiants  | CHECK           | YES      |
+--------------------+-------------------+-----------------+--------------+------------+-----------------+----------+
3 rows in set (0.07 sec)

mysql>
```

Mise en oeuvre - Ajout de contrainte CHECK

Exemple d'implémentation en SQL

```
ALTER TABLE etudiants ADD CONSTRAINT check_email CHECK (email LIKE "%_@__%.__%");
```

%: représente zéro ou plusieurs caractères
_: représente un seul caractère

Mise en oeuvre - Ajout de contrainte CHECK

```
c:\wamp64_1\bin\mysql\mysql8.0.31\bin\mysql.exe

mysql> alter table etudiants add constraint check_email CHECK (email LIKE '%_@__%.__%');
Query OK, 0 rows affected (0.60 sec)
Enregistrements: 0  Doublons: 0  Avertissements: 0

mysql> select * from information_schema.TABLE_CONSTRAINTS where TABLE_name='etudiants';
+--------------------+-------------------+-----------------+--------------+------------+-----------------+----------+
| CONSTRAINT_CATALOG | CONSTRAINT_SCHEMA | CONSTRAINT_NAME | TABLE_SCHEMA | TABLE_NAME | CONSTRAINT_TYPE | ENFORCED |
+--------------------+-------------------+-----------------+--------------+------------+-----------------+----------+
| def                | i3_bda            | email           | i3_bda       | etudiants  | UNIQUE          | YES      |
| def                | i3_bda            | PRIMARY         | i3_bda       | etudiants  | PRIMARY KEY     | YES      |
| def                | i3_bda            | check_email     | i3_bda       | etudiants  | CHECK           | YES      |
| def                | i3_bda            | etudiants_chk_1 | i3_bda       | etudiants  | CHECK           | YES      |
+--------------------+-------------------+-----------------+--------------+------------+-----------------+----------+
4 rows in set (0.00 sec)

mysql>
```

Mise en oeuvre - Ajout de contrainte UNIQUE

Exemple d’implémentation en SQL
ALTER TABLE etudiants ADD CONSTRAINT NIN UNIQUE (NIN);

Mise en oeuvre - Ajout de contrainte UNIQUE

```
c:\wamp64_1\bin\mysql\mysql8.0.31\bin\mysql.exe

mysql> ALTER TABLE etudiants ADD CONSTRAINT NIN UNIQUE (NIN);
Query OK, 0 rows affected (0.21 sec)
Enregistrements: 0  Doublons: 0  Avertissements: 0

mysql> select * from information_schema.TABLE_CONSTRAINTS where TABLE_name='etudiants';
+--------------------+--------------------+-----------------+--------------+------------+-----------------+----------+
| CONSTRAINT_CATALOG | CONSTRAINT_SCHEMA  | CONSTRAINT_NAME | TABLE_SCHEMA | TABLE_NAME | CONSTRAINT_TYPE | ENFORCED |
+--------------------+--------------------+-----------------+--------------+------------+-----------------+----------+
| def                | i3_bda             | email           | i3_bda       | etudiants  | UNIQUE          | YES      |
| def                | i3_bda             | NIN             | i3_bda       | etudiants  | UNIQUE          | YES      |
| def                | i3_bda             | PRIMARY         | i3_bda       | etudiants  | PRIMARY KEY     | YES      |
| def                | i3_bda             | check_email     | i3_bda       | etudiants  | CHECK           | YES      |
| def                | i3_bda             | etudiants_chk_1 | i3_bda       | etudiants  | CHECK           | YES      |
+--------------------+--------------------+-----------------+--------------+------------+-----------------+----------+
5 rows in set (0.00 sec)

mysql> _
```

Mise en oeuvre - Modification de contrainte

Exemple d’implémentation en SQL (Modification de contrainte à “note”)
ALTER TABLE etudiants DROP CONSTRAINT check_note; ALTER TABLE etudiants ADD CONSTRAINT check_note CHECK (note BETWEEN 0 AND 100);

Mise en oeuvre - Suppression de l'ancienne

```
c:\wamp64_1\bin\mysql\mysql8.0.31\bin\mysql.exe
mysql> select * from information_schema.TABLE_CONSTRAINTS where TABLE_name='etudiants';
+--------------------+--------------------+-----------------+--------------+------------+-----------------+----------+
| CONSTRAINT_CATALOG | CONSTRAINT_SCHEMA  | CONSTRAINT_NAME | TABLE_SCHEMA | TABLE_NAME | CONSTRAINT_TYPE | ENFORCED |
+--------------------+--------------------+-----------------+--------------+------------+-----------------+----------+
| def                | i3_bda             | email           | i3_bda       | etudiants  | UNIQUE          | YES      |
| def                | i3_bda             | NIN             | i3_bda       | etudiants  | UNIQUE          | YES      |
| def                | i3_bda             | PRIMARY         | i3_bda       | etudiants  | PRIMARY KEY     | YES      |
| def                | i3_bda             | check_email     | i3_bda       | etudiants  | CHECK           | YES      |
| def                | i3_bda             | etudiants_chk_1 | i3_bda       | etudiants  | CHECK           | YES      |
+--------------------+--------------------+-----------------+--------------+------------+-----------------+----------+
5 rows in set (0.00 sec)

mysql> alter table etudiants drop constraint etudiants_chk_1;
Query OK, 0 rows affected (0.30 sec)
Enregistrements: 0  Doublons: 0  Avertissements: 0

mysql> select * from information_schema.TABLE_CONSTRAINTS where TABLE_name='etudiants';
+--------------------+--------------------+-----------------+--------------+------------+-----------------+----------+
| CONSTRAINT_CATALOG | CONSTRAINT_SCHEMA  | CONSTRAINT_NAME | TABLE_SCHEMA | TABLE_NAME | CONSTRAINT_TYPE | ENFORCED |
+--------------------+--------------------+-----------------+--------------+------------+-----------------+----------+
| def                | i3_bda             | email           | i3_bda       | etudiants  | UNIQUE          | YES      |
| def                | i3_bda             | NIN             | i3_bda       | etudiants  | UNIQUE          | YES      |
| def                | i3_bda             | PRIMARY         | i3_bda       | etudiants  | PRIMARY KEY     | YES      |
| def                | i3_bda             | check_email     | i3_bda       | etudiants  | CHECK           | YES      |
+--------------------+--------------------+-----------------+--------------+------------+-----------------+----------+
4 rows in set (0.00 sec)

mysql>
```

Mise en oeuvre - Ajout de la nouvelle

c:\wamp64_1\bin\mysql\mysql8.0.31\bin\mysql.exe

```
mysql> select * from information_schema.TABLE_CONSTRAINTS where TABLE_name='etudiants';
+--------------------+--------------------+-----------------+--------------+------------+-----------------+----------+
| CONSTRAINT_CATALOG | CONSTRAINT_SCHEMA  | CONSTRAINT_NAME | TABLE_SCHEMA | TABLE_NAME | CONSTRAINT_TYPE | ENFORCED |
+--------------------+--------------------+-----------------+--------------+------------+-----------------+----------+
| def                | i3_bda             | email           | i3_bda       | etudiants  | UNIQUE          | YES      |
| def                | i3_bda             | NIN             | i3_bda       | etudiants  | UNIQUE          | YES      |
| def                | i3_bda             | PRIMARY         | i3_bda       | etudiants  | PRIMARY KEY     | YES      |
| def                | i3_bda             | check_email     | i3_bda       | etudiants  | CHECK           | YES      |
+--------------------+--------------------+-----------------+--------------+------------+-----------------+----------+
4 rows in set (0.00 sec)

mysql> alter table etudiants add constraint check_note CHECK (note BETWEEN 0 AND 100);
Query OK, 0 rows affected (0.27 sec)
Enregistrements: 0  Doublons: 0  Avertissements: 0

mysql> select * from information_schema.TABLE_CONSTRAINTS where TABLE_name='etudiants';
+--------------------+--------------------+-----------------+--------------+------------+-----------------+----------+
| CONSTRAINT_CATALOG | CONSTRAINT_SCHEMA  | CONSTRAINT_NAME | TABLE_SCHEMA | TABLE_NAME | CONSTRAINT_TYPE | ENFORCED |
+--------------------+--------------------+-----------------+--------------+------------+-----------------+----------+
| def                | i3_bda             | email           | i3_bda       | etudiants  | UNIQUE          | YES      |
| def                | i3_bda             | NIN             | i3_bda       | etudiants  | UNIQUE          | YES      |
| def                | i3_bda             | PRIMARY         | i3_bda       | etudiants  | PRIMARY KEY     | YES      |
| def                | i3_bda             | check_email     | i3_bda       | etudiants  | CHECK           | YES      |
| def                | i3_bda             | check_note      | i3_bda       | etudiants  | CHECK           | YES      |
+--------------------+--------------------+-----------------+--------------+------------+-----------------+----------+
5 rows in set (0.00 sec)

mysql>
```

1. Précautions : Assurez-vous que la modification n'entraîne pas de perte de données ou d'incohérence dans votre base de données.
2. Droits d'accès : Vous aurez besoin des droits d'accès appropriés pour modifier les contraintes.
3. SGBD : La syntaxe peut varier légèrement selon le SGBD que vous utilisez (MySQL, PostgreSQL, Oracle, etc.).

Les CI dynamiques (CID)

Introduction

1. Elles se réfèrent à des règles qui s'appliquent en temps réel, lors des opérations sur les données.

2. Elles s'adaptent aux données elles-mêmes et permettent une validation plus contextuelle et complexe.

Exemple, une CIS peut être utilisée pour s'assurer qu'un salaire inséré est supérieur à zéro, Alors qu'un CID peut automatiquement mettre à jour une date de modification chaque fois qu'une ligne est modifiée.

Introduction

1. Leur 'importance réside dans leur capacité à renforcer la qualité des données.

2. Elles permettent non seulement de prévenir les erreurs de saisie, mais aussi d'automatiser des processus plus complexes.

3. Leur utilisation nécessite une attention particulière, car une mauvaise implémentation peut générer des problèmes de performance.

CI dynamique par déclencheur

Notion de déclencheurs (ou triggers en anglais):

1. C’est des objets de BD qui sont automatiquement exécutés (ou "déclenchés") en réponse à des événements spécifiques (**INSERT**, **UPDATE**, **DELETE**) sur une table ou une vue.

2. L’exécution peut être lancée avant (**BEFORE**) ou après (**AFTER**) l’exécution de l’opération.

CI dynamique par déclencheur - Exemple

Soit une table utilisateurs où nous souhaitons nous assurer que l'âge d'un utilisateur est supérieur à 18 ans lors de l'insertion ou de la mise à jour d'un enregistrement.

Exemple d'implémentation en SQL - Création de la table

```sql
CREATE TABLE utilisateur (
    id INT PRIMARY KEY,
    nom VARCHAR(50) NOT NULL,
    age INT
);
```

CI dynamique par déclencheur - Exemple

Exemple d'implémentation en SQL - Création du déclencheur

```
CREATE TRIGGER validation_age
BEFORE INSERT OR UPDATE ON utilisateur
FOR EACH ROW
BEGIN
    IF NEW.age < 18 THEN
        SIGNAL SQLSTAT '45000'
        SET MESSAGE_TEXT = 'L\'âge doit être supérieur ou égale à 18 ans';
    END IF;
END;
```

CI dynamique par déclencheur - Exemple

Notion	explication
CREATE TRIGGER	C'est la commande utilisée pour créer un nouveau déclencheur.
validation_age	C'est le nom du déclencheur. Ce nom doit être unique dans la base de données.
BEFORE	Indique que le déclencheur doit s'exécuter avant l'opération spécifiée (dans ce cas, avant l'insertion ou la mise à jour).
INSERT OR UPDATE	Cela signifie que le déclencheur sera activé soit lors d'une insertion de nouvelles données, soit lors d'une mise à jour de données existantes dans la table.
ON utilisateurs	Indique que ce déclencheur est associé à la table utilisateurs.

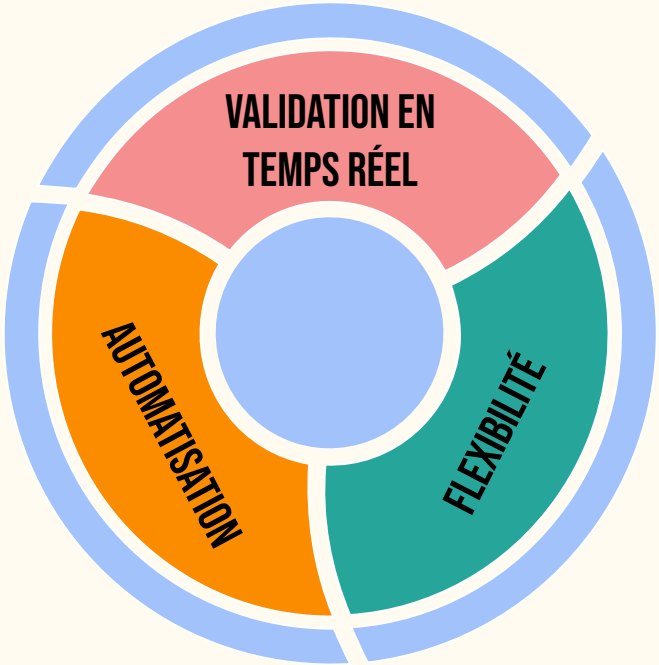
CI dynamique par déclencheur - Exemple

Notion	explication
FOR EACH ROW	Le déclencheur s'exécutera pour chaque ligne affectée par l'opération. Par exemple, si plusieurs lignes sont insérées ou mises à jour en une seule commande, le déclencheur sera exécuté pour chacune de ces lignes.
BEGIN et END	Délimiteurs du bloc de code.
IF	C’est le SI conditionnel
NEW.age < 18	**NEW** fait référence aux nouvelles valeurs qui seront insérées ou mises à jour.

CI dynamique par déclencheur - Exemple

Notion	explication
SIGNAL	Cette commande est utilisée pour générer une erreur personnalisée dans la base de données.
SQLSTATE '45000'	C'est un code d'état SQL qui indique une erreur générée par l'utilisateur. Le code 45000 est souvent utilisé pour des erreurs personnalisées.
SET MESSAGE_TEXT	Cette commande définit le message d'erreur qui sera renvoyé lorsque le déclencheur est activé.

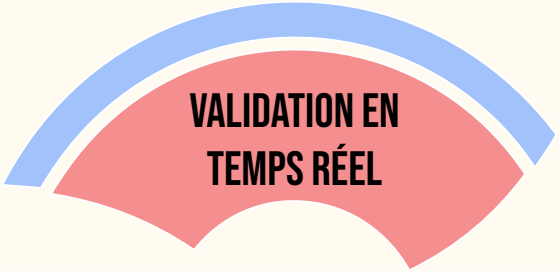
Caractéristiques des CI dynamique



Caractéristiques des CI dynamique

- Les CI dynamiques permettent de valider les données au moment où elles sont manipulées, garantissant ainsi que seules les données conformes aux règles définies sont acceptées.

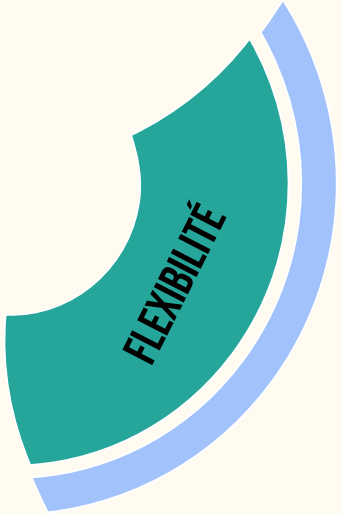
- À chaque fois qu'une opération d'insertion ou de mise à jour est effectuée, la BD vérifie immédiatement si les données respectent les règles définies.



Caractéristiques des CI dynamique

- Elles peuvent être adaptées pour répondre à des besoins spécifiques, en fonction des valeurs des données et des contextes d'utilisation.

- Elles sont utilisées là où les CI statiques ne peuvent s'appliquer.



Caractéristiques des CI dynamique

01	RÉACTION AUX ÉVÉNEMENTS	• Réagir automatiquement à des événements. • Aucune réaction manuelle.
02	EXÉCUTION CONDITIONNELLE	• Contenir des conditions qui déterminent si une action (mise à jour, générer une alerte) doit être effectuée ou non.
03	CENTRALISATION DE LA LOGIQUE MÉTIER	• Logiques métier centralisé dans la BD • Réduction de la duplication du code dans les applications qui utilise la BD.
04	ADAPTABILITÉ	• Modification ou ajout facile pour s'adapter à de nouveaux besoins sans nécessiter des changements majeurs dans l'application.
05	GESTION DES HISTORIQUE ET AUDITS	• Auto-enregistrement des modifications dans des tables d'historique ou d'audit. • Traçabilité des données.

Caractéristiques des CI dynamique

Exemple de flexibilité:

Imaginons une table commandes où nous souhaitons appliquer différentes règles selon le statut de la commande :

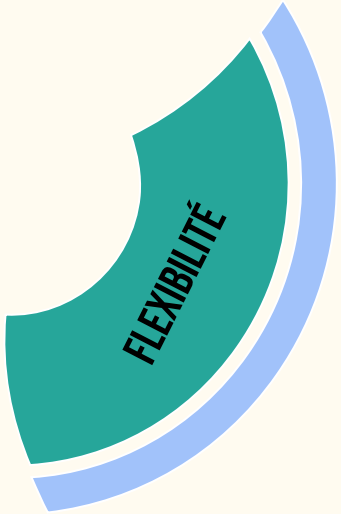
1. Si une commande est ***annulée***, un déclencheur pourrait automatiquement *mettre à jour le stock* du produit concerné.
2. Si une commande est ***confirmée***, un autre déclencheur pourrait *envoyer une notification* au client.

Caractéristiques des CI dynamique

Exemple de flexibilité:

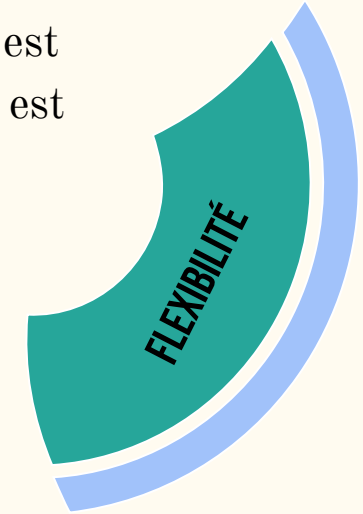
Exemple d'implémentation en SQL - Création de la table Commandes

```sql
CREATE TABLE commandes (
    id INT PRIMARY KEY,
    client_id INT,
    produit_id INT,
    quantite INT,
    statut VARCHAR(20), -- Peut être 'confirmée', 'annulée', 'expédiée'
    date_commande DATETIME
) ENGINE=InnoDB;
```



Caractéristiques des CI dynamique

<u>Scénario d'annulation de la commande:</u> Lorsqu'une commande est annulée, on ***mis à jour*** le stock du produit concerné. Le déclencheur est exécuté ***avant*** la mise à jour de la commande.

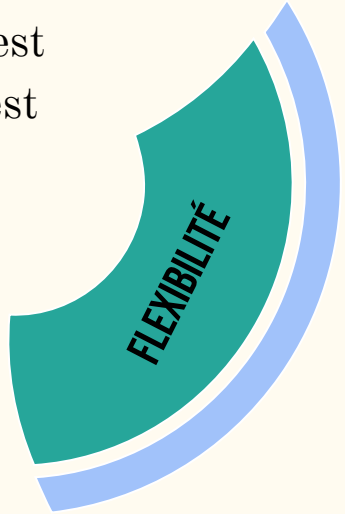


Exemple d'implémentation en SQL - Création du déclencheur d'annulation

```
CREATE TRIGGER gerer_annulation
BEFORE UPDATE ON commandes
FOR EACH ROW
BEGIN
    IF NEW.statut = 'annulée' AND  OLD.statut != 'annulée' THEN
        UPDATE produits
        SET stock = stock + OLD.quantité WHERE id = OLD.produit_id;
    END IF;
END;
```

Caractéristiques des CI dynamique

<u>Scénario de confirmation de la commande:</u> Lorsqu'une commande est confirmée, on ***envoi une notification*** au client. Le déclencheur est exécuté ***après*** la modification de la commande.



Exemple d'implémentation en SQL - Création du déclencheur de confirmation

```
CREATE TRIGGER gerer_confirmation
AFTER UPDATE ON commandes
FOR EACH ROW
BEGIN
    IF NEW.statut = 'confirmée' AND  OLD.statut != 'confirmée' THEN
            INSERT INTO notifications (client_id, message, date_notification)
            VALUES (New.client_id, 'Votre commande a été confirmée.', NOW());
    END IF;
END;
```

Caractéristiques des CI dynamique

- La capacité d'un SGBD à appliquer automatiquement des règles et des restrictions sur les données en temps réel, sans intervention manuelle.

- Grâce à des déclencheurs (triggers) et des règles de validation, les contraintes dynamiques peuvent automatiser des processus tels que la mise à jour de champs ou la vérification de formats, réduisant ainsi le risque d'erreurs humaines.

Caractéristiques des CI dynamique

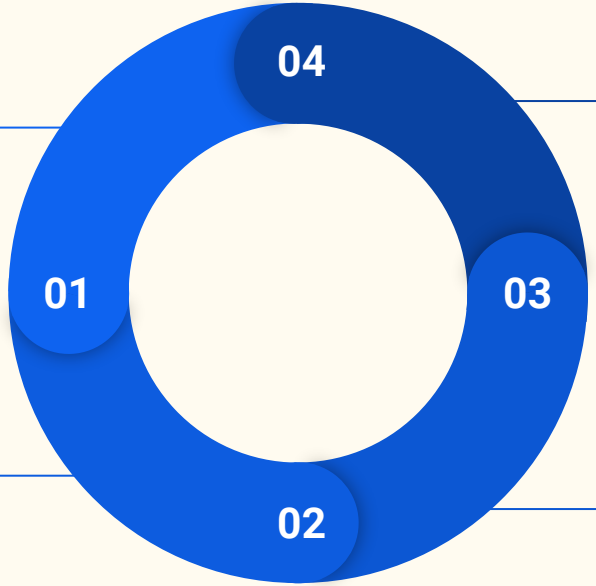
Avantages:

COHÉRENCE DES DONNÉES

Erreurs humaines réduites, car les règles sont appliquées automatiquement par le SGBD.

VALIDATION EN TEMPS RÉEL

Les données sont vérifiées à chaque opération, garantissant ainsi leur validité immédiatement.



RÉDUCTION DES COÛTS

Moins d'erreurs entraînent moins de coûts liés à la correction des données et à la gestion des anomalies.

MAINTENANCE SIMPLIFIÉE

Aucune écriture de code supplémentaire pour gérer les règles d'intégrité, car elles sont intégrées au SGBD.

Caractéristiques des CI dynamique

Exemple d'automatisation:

Considérons la même table commande et nous allons créer un déclencheur qui s'exécute ***avant*** chaque mise à jour dans la table commandes. Ce déclencheur ***mettra automatiquement à jour*** le champ date_modification avec l'heure actuelle.

Exemple d'implémentation en SQL - Création de la table Commandes

```
CREATE TABLE commandes (
   id INT PRIMARY KEY,
   client_id INT,
   produit_id INT,
   quantite INT,
   statut VARCHAR(20), -- Peut être 'confirmée', 'annulée', 'expédiée'
   date_commande DATETIME
) ENGINE=InnoDB;
```

Caractéristiques des CI dynamique

Déclencheur de mise à jour de la date de modification

Exemple d'implémentation en SQL - Création du déclencheur de mise à jour de la date

```
CREATE TRIGGER mise_a_jour_date_modification
BEFORE UPDATE ON commandes
FOR EACH ROW
BEGIN
    SET NEW.date_modification = NOW();
END;
```

Caractéristiques des CI dynamique

Scénario d'automatisation:

➔ Insertion d'une commande le 29/10/2024 à 13h15mn44s:

Exemple d'implémentation en SQL

```
INSERT INTO commandes (id, client_id, montant, date_modification)
VALUES (1, 101, 250.00, '2024-10-29 13:15:44');
```

➔ Mise à jour de la commande:

Exemple d'implémentation en SQL

```
UPDATE commandes
SET montant = 300.00 WHERE id = 1;
```

Cette mise à jour déclenche le trigger qui met à jour le champs date_modification avec la date et l'heure du jour.

Importance des CI Dynamiques

1. **Cohérence des Données (Validation Continue) :** Les données restent cohérentes tout au long de leur cycle de vie.
2. **Intégrité Référentielle (Relations entre Tables) :** Assurent que les relations entre différentes tables sont maintenues.
3. **Prévention des erreurs (Réduction des erreurs Humaines) :** Les SGBD minimisent le risque d'erreurs humaines lors de la saisie ou de la modification des données.

Importance des CI Dynamiques

4. **Amélioration de la Qualité des Données :** Contribuent à garantir que les données sont précises, complètes et fiables, ce qui est essentiel pour une prise de décision efficace.
5. **Facilitation de la Maintenance :** Simplifient la gestion des données, car les développeurs n'ont pas besoin de mettre en œuvre des validations manuelles dans leur code.
6. **Performance Optimisée :** Les SGBD optimisent les vérifications des contraintes, ce qui améliorer les performances des opérations de BD en évitant des validations redondantes.

Importance des CI Dynamiques

7. **Conformité Réglementaire (Respect des Normes) :** Aident les organisations à se conformer à ces exigences de respect des normes de qualité des données.
8. **Historique et Audit (Traçabilité des Modifications) :** Peuvent également faciliter la création de journaux d'audit, permettant de suivre les modifications apportées aux données et d'identifier les erreurs ou les abus.

Conclusion

- Les contraintes d'intégrité dynamique sont essentielles pour maintenir la *qualité*, la *cohérence* et la *fiabilité* des données dans un SGBD.
- Elles permettent de *prévenir les erreurs* et *d'assurer l'intégrité* des données, *facilitent la maintenance* et *garantissent la conformité* aux normes réglementaires.
- C'est un fondement crucial pour toute application qui repose sur des données fiables.